

RELATÓRIO DO TRABALHO PRATICO

Simulador de Estruturas de Dados Lineares em C

Integrantes

Samuel Sampaio Lins

Kauan Moreira

Disciplina: Estrutura de Dados

Relatório de entrega do sistema desenvolvido em linguagem C

Sumário

- 1. Introdução
- 2. Objetivo do sistema
- 3. Descrição geral do funcionamento
- 4. Estruturas implementadas
- 5. Organização do código-fonte
- 6. Principais dificuldades encontradas
- 7. Prints e evidências da execução
- 8. Divisão das tarefas entre os membros
- 9. Como compilar e executar
- 10. Considerações finais

1. Introdução

Este relatório apresenta o desenvolvimento do Simulador de Estruturas de Dados Lineares em C. O sistema foi criado para demonstrar, por meio de menus interativos, o funcionamento das principais estruturas lineares estudadas na disciplina: lista sequencial, lista simplesmente encadeada, pilha, fila, fila circular, fila dupla e fila de prioridade.

A proposta do trabalho foi implementada com foco em organização do código, separação das estruturas em arquivos próprios, uso de funções para cada operação e validações para evitar operações inválidas, como remover elementos de estruturas vazias.

2. Objetivo do sistema

O objetivo do sistema é permitir que o usuário escolha uma estrutura de dados e execute operações compatíveis com suas regras. O programa apresenta um menu principal com todas as estruturas e, ao selecionar uma delas, exibe um submenu específico com as opções de inserção, remoção, consulta, exibição, verificação de vazio e quantidade de elementos, quando aplicável.

- Permitir a simulação prática de sete estruturas de dados lineares.
- Manter os dados de cada estrutura separados e independentes.
- Limitar cada estrutura a no máximo 100 elementos, conforme solicitado.
- Utilizar funções para organizar as operações e facilitar a manutenção do código.
- Tratar erros comuns de uso, como posições inválidas, estrutura cheia e estrutura vazia.

3. Descrição geral do funcionamento

Ao iniciar o programa, o usuário visualiza o menu principal do simulador. Cada opção direciona para o submenu de uma estrutura. A partir desse submenu, o usuário pode inserir, remover, consultar e exibir elementos. O elemento padrão usado pelo sistema possui ID, nome e prioridade. O campo prioridade é usado principalmente na fila de prioridade, mas foi mantido no tipo padrão para simplificar o reaproveitamento das funções auxiliares.

O programa foi desenvolvido para que o usuário possa retornar ao menu principal sem encerrar a execução. Dessa forma, é possível testar várias estruturas em uma única execução. Cada estrutura

possui sua própria variável de armazenamento, por isso os dados inseridos em uma pilha, por exemplo, não interferem nos dados de uma fila ou lista.

4. Estruturas implementadas

Lista sequencial simples com vetor

Foi implementada com um vetor de até 100 posições. Permite inserir em uma posição escolhida, remover por posição, buscar por ID, listar elementos e informar a quantidade. Quando ocorre inserção ou remoção no meio da lista, os elementos são deslocados para manter a ordem.

Lista simplesmente encadeada

Foi implementada com ponteiros e alocação dinâmica. Cada elemento fica dentro de um nó, e cada nó aponta para o próximo. Permite inserir no início, inserir no fim, remover por ID, buscar por ID e listar os elementos.

Pilha

Segue a regra LIFO, isto é, o último elemento inserido é o primeiro a ser removido. Foi implementada com vetor e uma variável topo, que indica a posição do último elemento armazenado.

Fila

Segue a regra FIFO, isto é, o primeiro elemento inserido é o primeiro a ser removido. Foi implementada com controle de início, fim e quantidade, permitindo enfileirar, desenfileirar, consultar o início e exibir a fila.

Fila circular

Também segue FIFO, mas usa aritmética modular para reaproveitar posições do vetor. Os índices início e fim avançam de forma circular, retornando ao início do vetor quando chegam ao final.

Fila dupla

Permite inserções e remoções tanto no início quanto no fim. Essa estrutura é mais flexível que a fila comum, pois permite operações nos dois extremos.

Fila de prioridade

Armazena elementos considerando o campo prioridade. A remoção sempre retira o elemento de maior prioridade. A estrutura foi mantida ordenada da maior para a menor prioridade para facilitar a consulta do próximo elemento a ser removido.

5. Organização do código-fonte

O código foi organizado em arquivos separados para facilitar a leitura, a manutenção e a explicação durante a apresentação. Os arquivos .h contêm as declarações das estruturas e funções, enquanto os arquivos .c contêm a implementação das operações.

Arquivo	Função no projeto
main.c	Contém o menu principal, os submenus e as chamadas para as operações de cada estrutura.
elemento.h	Define o tipo Elemento, contendo ID, nome e

	prioridade.
utils.c / utils.h	Contem funções auxiliares de leitura, validacao de entrada e exibição de elementos.
lista_sequencial.c / .h	Implementa a lista sequencial com vetor.
lista_encadeada.c / .h	Implementa a lista simplesmente encadeada com ponteiros e alocação dinâmica.
pilha.c / .h	Implementa a pilha com regra LIFO.
fila.c / .h	Implementa a fila comum com regra FIFO.
fila_circular.c / .h	Implementa a fila circular com controle de início e fim.
fila_dupla.c / .h	Implementa a fila dupla, com operações nos dois extremos.
fila_prioridade.c / .h	Implementa a fila de prioridade, ordenada por prioridade.
Makefile	Facilita a compilação do projeto por meio do comando make.

6. Principais dificuldades encontradas

Durante o desenvolvimento, algumas partes exigiram mais atenção por envolverem controle de índices, ponteiros e validações. As principais dificuldades foram:

- Controlar corretamente os índices início e fim na fila circular, evitando perda de dados e reaproveitando as posições do vetor.
- Implementar a lista simplesmente encadeada com alocação dinâmica, pois foi necessário manipular ponteiros e atualizar ligacoes entre os nos.
- Garantir que cada estrutura mantivesse seus próprios dados, sem interferir no funcionamento das outras estruturas.
- Criar validações para impedir remoção ou consulta em estruturas vazias.
- Tratar a situação de estrutura cheia nas implementacoes que usam vetor com limite de 100 elementos.
- Organizar o projeto em varios arquivos .c e .h, mantendo as responsabilidades de cada arquivo bem separadas.
- Testar as operações principais de cada estrutura para verificar se as regras LIFO, FIFO e prioridade estavam sendo respeitadas.

7. Prints e evidencias da execução

Os prints abaixo foram obtidos a partir de uma execução de teste do programa no terminal. Eles demonstram o menu principal, as operações principais das estruturas e a compilação do projeto.

Execucao - Menu principal

```
===== SIMULADOR DE ESTRUTURAS DE DADOS =====  
1 - Lista Sequencial  
2 - Lista Simplesmente Encadeada  
3 - Pilha  
4 - Fila  
5 - Fila Circular  
6 - Fila Dupla  
7 - Fila de Prioridade  
0 - Sair
```

Print 1 - Menu principal

Execucao - Lista sequencial

```
===== MENU DA LISTA SEQUENCIAL =====  
1 - Inserir elemento em uma posicao  
2 - Remover elemento de uma posicao  
3 - Buscar elemento por ID  
4 - Exibir estrutura  
5 - Verificar se esta vazia  
6 - Verificar quantidade de elementos  
7 - Verificar se esta cheia  
0 - Voltar ao menu principal  
Escolha uma opcao: ID: Nome: Posicao para inserir (comecando em 1): Elemento inserido com  
sucesso.
```

```
===== MENU DA LISTA SEQUENCIAL =====  
1 - Inserir elemento em uma posicao  
2 - Remover elemento de uma posicao  
3 - Buscar elemento por ID  
4 - Exibir estrutura  
5 - Verificar se esta vazia  
6 - Verificar quantidade de elementos  
7 - Verificar se esta cheia  
0 - Voltar ao menu principal  
Escolha uma opcao:
```

```
===== LISTA SEQUENCIAL =====  
Posicao 1 -> ID: 10 | Nome: Alice
```

```
===== MENU DA LISTA SEQUENCIAL =====  
1 - Inserir elemento em uma posicao  
2 - Remover elemento de uma posicao  
3 - Buscar elemento por ID  
4 - Exibir estrutura  
5 - Verificar se esta vazia  
6 - Verificar quantidade de elementos  
...
```

```
Execucao - Lista encadeada

===== MENU DA LISTA SIMPLEMENTE ENCADEADA =====
1 - Inserir elemento no inicio
2 - Inserir elemento no fim
3 - Remover elemento por ID
4 - Buscar elemento por ID
5 - Exibir estrutura
6 - Verificar se esta vazia
7 - Verificar quantidade de elementos
8 - Verificar se esta cheia
0 - Voltar ao menu principal
Escolha uma opcao: ID: Nome: Elemento inserido no inicio com sucesso.

===== MENU DA LISTA SIMPLEMENTE ENCADEADA =====
1 - Inserir elemento no inicio
2 - Inserir elemento no fim
3 - Remover elemento por ID
4 - Buscar elemento por ID
5 - Exibir estrutura
6 - Verificar se esta vazia
7 - Verificar quantidade de elementos
8 - Verificar se esta cheia
0 - Voltar ao menu principal
Escolha uma opcao: ID: Nome: Elemento inserido no fim com sucesso.

===== MENU DA LISTA SIMPLEMENTE ENCADEADA =====
1 - Inserir elemento no inicio
2 - Inserir elemento no fim
3 - Remover elemento por ID
4 - Buscar elemento por ID
5 - Exibir estrutura
6 - Verificar se esta vazia
7 - Verificar quantidade de elementos
...
```

Execucao - Pilha e fila comum

===== MENU DA PILHA =====

- 1 - Empilhar elemento
- 2 - Desempilhar elemento
- 3 - Consultar topo
- 4 - Exibir estrutura
- 5 - Verificar se esta vazia
- 6 - Verificar quantidade de elementos
- 7 - Verificar se esta cheia
- 0 - Voltar ao menu principal

Escolha uma opcao: ID: Nome: Elemento empilhado com sucesso.

===== MENU DA PILHA =====

- 1 - Empilhar elemento
- 2 - Desempilhar elemento
- 3 - Consultar topo
- 4 - Exibir estrutura
- 5 - Verificar se esta vazia
- 6 - Verificar quantidade de elementos
- 7 - Verificar se esta cheia
- 0 - Voltar ao menu principal

Escolha uma opcao: Topo da pilha:

ID: 30 | Nome: Topo | Prioridade: 0

===== MENU DA PILHA =====

- 1 - Empilhar elemento
- 2 - Desempilhar elemento
- 3 - Consultar topo
- 4 - Exibir estrutura
- 5 - Verificar se esta vazia
- 6 - Verificar quantidade de elementos
- 7 - Verificar se esta cheia
- 0 - Voltar ao menu principal

...

Print 4 - Pilha e Fila: regras LIFO e FIFO

Execucao - Fila circular

===== MENU DA FILA CIRCULAR =====

- 1 - Enfileirar elemento
- 2 - Desenfileirar elemento
- 3 - Consultar inicio
- 4 - Consultar fim
- 5 - Exibir estrutura
- 6 - Verificar se esta cheia
- 7 - Verificar se esta vazia
- 8 - Verificar quantidade de elementos
- 0 - Voltar ao menu principal

Escolha uma opcao: ID: Nome: Elemento enfileirado com sucesso.

===== MENU DA FILA CIRCULAR =====

- 1 - Enfileirar elemento
- 2 - Desenfileirar elemento
- 3 - Consultar inicio
- 4 - Consultar fim
- 5 - Exibir estrutura
- 6 - Verificar se esta cheia
- 7 - Verificar se esta vazia
- 8 - Verificar quantidade de elementos
- 0 - Voltar ao menu principal

Escolha uma opcao: ID: Nome: Elemento enfileirado com sucesso.

===== MENU DA FILA CIRCULAR =====

- 1 - Enfileirar elemento
- 2 - Desenfileirar elemento
- 3 - Consultar inicio
- 4 - Consultar fim
- 5 - Exibir estrutura
- 6 - Verificar se esta cheia
- 7 - Verificar se esta vazia

...

Print 5 - Fila circular: consulta e reaproveitamento de posições

Execucao - Fila dupla

===== MENU DA FILA DUPLA =====

- 1 - Inserir no inicio
- 2 - Inserir no fim
- 3 - Remover do inicio
- 4 - Remover do fim
- 5 - Consultar inicio
- 6 - Consultar fim
- 7 - Exibir estrutura
- 8 - Verificar se esta vazia
- 9 - Verificar quantidade de elementos
- 10 - Verificar se esta cheia
- 0 - Voltar ao menu principal

Escolha uma opcao: ID: Nome: Elemento inserido no inicio com sucesso.

===== MENU DA FILA DUPLA =====

- 1 - Inserir no inicio
- 2 - Inserir no fim
- 3 - Remover do inicio
- 4 - Remover do fim
- 5 - Consultar inicio
- 6 - Consultar fim
- 7 - Exibir estrutura
- 8 - Verificar se esta vazia
- 9 - Verificar quantidade de elementos
- 10 - Verificar se esta cheia
- 0 - Voltar ao menu principal

Escolha uma opcao: ID: Nome: Elemento inserido no fim com sucesso.

===== MENU DA FILA DUPLA =====

- 1 - Inserir no inicio
- 2 - Inserir no fim
- 3 - Remover do inicio

...

Print 6 - Fila dupla: operações nos dois extremos

```
●●● Execução - Fila de prioridade

===== MENU DA FILA DE PRIORIDADE =====
1 - Inserir elemento com prioridade
2 - Remover elemento de maior prioridade
3 - Consultar próximo elemento a ser removido
4 - Exibir fila ordenada por prioridade
5 - Verificar se esta vazia
6 - Verificar quantidade de elementos
7 - Verificar se esta cheia
0 - Voltar ao menu principal
Escolha uma opção: ID: Nome: Prioridade: Elemento inserido com prioridade com sucesso.

===== MENU DA FILA DE PRIORIDADE =====
1 - Inserir elemento com prioridade
2 - Remover elemento de maior prioridade
3 - Consultar próximo elemento a ser removido
4 - Exibir fila ordenada por prioridade
5 - Verificar se esta vazia
6 - Verificar quantidade de elementos
7 - Verificar se esta cheia
0 - Voltar ao menu principal
Escolha uma opção: ID: Nome: Prioridade: Elemento inserido com prioridade com sucesso.

===== MENU DA FILA DE PRIORIDADE =====
1 - Inserir elemento com prioridade
2 - Remover elemento de maior prioridade
3 - Consultar próximo elemento a ser removido
4 - Exibir fila ordenada por prioridade
5 - Verificar se esta vazia
6 - Verificar quantidade de elementos
7 - Verificar se esta cheia
0 - Voltar ao menu principal
Escolha uma opção:
...
```

Print 7 - Fila de prioridade: ordenação por prioridade

```
●●● Compilação - make

gcc -Wall -Wextra -std=c99 -pedantic -c main.c
gcc -Wall -Wextra -std=c99 -pedantic -c utils.c
gcc -Wall -Wextra -std=c99 -pedantic -c lista_sequencial.c
gcc -Wall -Wextra -std=c99 -pedantic -c lista_encadeada.c
gcc -Wall -Wextra -std=c99 -pedantic -c pilha.c
gcc -Wall -Wextra -std=c99 -pedantic -c fila.c
gcc -Wall -Wextra -std=c99 -pedantic -c fila_circular.c
gcc -Wall -Wextra -std=c99 -pedantic -c fila_dupla.c
gcc -Wall -Wextra -std=c99 -pedantic -c fila_prioridade.c
gcc -Wall -Wextra -std=c99 -pedantic -o simulador main.o utils.o lista_sequencial.o
lista_encadeada.o pilha.o fila.o fila_circular.o fila_dupla.o fila_prioridade.o
```

Print 8 - Compilação do projeto com Makefile

8. Divisão das tarefas entre os membros

A divisão das tarefas foi feita de forma equilibrada para que os dois integrantes participassem do desenvolvimento e entendessem o funcionamento geral do sistema.

Integrante	Tarefas realizadas
Samuel Sampaio Lins	Organização do menu principal e dos submenus; implementação e revisao da lista sequencial e da pilha; testes de inserção, remoção, consulta e exibição; apoio na montagem do relatório e revisao final do funcionamento.
Kauan Moreira	Implementação e revisao da lista simplesmente encadeada, fila comum, fila circular, fila dupla e fila de prioridade; apoio nas validações de estrutura vazia e cheia; testes das regras FIFO, fila circular e prioridade; revisao da organização dos arquivos .c e .h.

9. Como compilar e executar

Para compilar o projeto, basta abrir o terminal dentro da pasta do código-fonte e executar:

make

Caso o make não esteja disponível, também é possível compilar diretamente com gcc:

```
gcc -Wall -Wextra -std=c99 -pedantic -o simulador main.c utils.c lista_sequencial.c  
lista_encadeada.c pilha.c fila.c fila_circular.c fila_dupla.c fila_prioridade.c
```

Para executar o sistema no Linux, macOS, Git Bash ou WSL:

./simulador

No Windows, usando MinGW, o executável pode ser iniciado com:

simulador.exe

10. Considerações finais

O projeto atingiu o objetivo de simular as principais estruturas de dados lineares em linguagem C. O sistema permite que o usuário escolha uma estrutura, realize operações principais e visualize o comportamento de cada uma delas. A separação em arquivos e o uso de funções contribuem para deixar o código mais organizado e facilitar a explicação durante a apresentação.

As validações implementadas ajudam a evitar erros durante a execução, principalmente em casos de estrutura vazia, estrutura cheia e posições inválidas. Com isso, o simulador demonstra na prática os conceitos estudados na disciplina, como LIFO, FIFO, ponteiros, alocação dinâmica, controle de índices e prioridade.